

MALWARE ANALYSIS REPORT

BillingApp.exe — Static Analysis & Safe Lab Procedure

Field	Value
Sample	BillingApp.exe
Size	28,216,596 bytes (~27 MiB)
SHA-256	a7664b8ce3807c15a1e76384d6312eead5f5df3140c48a74dc7298e09f7acce7
MD5	5094d0ce132488b040e37a0503687963
Format	PE32+ / x86-64 / Windows CUI
Static verdict	No clear conventional malware behavior observed; application weaknesses identified
Execution status	NOT EXECUTED during this analysis

1. Executive Summary

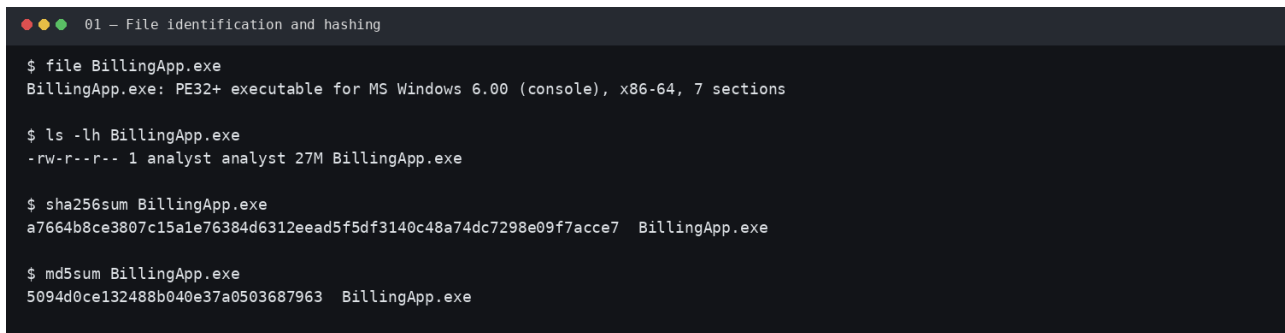
BillingApp.exe was analyzed as an untrusted Windows executable using non-executing static-analysis techniques. It is a 64-bit PE packaged with PyInstaller and embeds Python 3.14, Flask, SQLAlchemy, SQLite, ReportLab, Waitress and a React static frontend. Static extraction exposed billing APIs, local server behavior, a hard-coded Flask secret, and a default admin/admin123 credential. No hard-coded external C2 URL/IP or clear persistence mechanism was identified in the reviewed static evidence. Dynamic analysis remains necessary for a final runtime verdict.

2. Safety and Lab Setup

Use a disposable VMware snapshot, host-only/internal networking, no shared clipboard/folders, no corporate credentials, and no bridged networking. Do not execute an unknown sample on the analyst workstation.

3. File Identification and Hashing

Commands: `file BillingApp.exe`; `sha256sum BillingApp.exe`; `md5sum BillingApp.exe`. The primary SHA-256 is `a7664b8ce3807c15a1e76384d6312eead5f5df3140c48a74dc7298e09f7acce7`.

A terminal window titled "01 - File identification and hashing" displays the following commands and their outputs:

```
$ file BillingApp.exe
BillingApp.exe: PE32+ executable for MS Windows 6.00 (console), x86-64, 7 sections

$ ls -lh BillingApp.exe
-rw-r--r-- 1 analyst analyst 27M BillingApp.exe

$ sha256sum BillingApp.exe
a7664b8ce3807c15a1e76384d6312eead5f5df3140c48a74dc7298e09f7acce7 BillingApp.exe

$ md5sum BillingApp.exe
5094d0ce132488b040e37a0503687963 BillingApp.exe
```

4. PE Header and Section Analysis

objdump -x BillingApp.exe reveals PE32+, x86-64, Windows CUI, seven sections, and an entry point at 0xDCf0. Section entropy is not by itself a malware indicator.

```
02 - PE headers and sections

$ objdump -x BillingApp.exe | grep -E "Magic|Time/Date|AddressOfEntryPoint|Subsystem"
Magic                020b  (PE32+)
Time/Date             Sat Sep 19 11:37:24 2026
AddressOfEntryPoint   000000000000dcf0
Subsystem             00000003  (Windows CUI)

Idx  Name      Raw Size  Virtual Size  Entropy
0    .text     210432    210064        6.490
1    .rdata     83968     83854        5.724
2    .data      3584      19192        1.809
3    .pdata     10240     10212        5.501
4    .fptable    512        256         0.000
5    .rsrc      61440     61324        7.350
6    .reloc      2048       1888        5.244
```

5. Import Analysis

Imports are USER32.dll, KERNEL32.dll and ADVAPI32.dll. Process/file/debugging APIs occur in the PyInstaller/Python runtime. They should be interpreted in context.

03 - Imported Windows APIs

```
$ objdump -p BillingApp.exe | sed -n "/The Import Tables/,/The Function Table/p"
```

```
DLL Name: USER32.dll
```

```
GetCursorPos, CreateWindowExW, ShowWindow, GetMessageW, DispatchMessageW
```

```
DLL Name: KERNEL32.dll
```

```
CreateProcessW, OpenProcess, Process32FirstW/NextW, LoadLibraryExW
```

```
CreateFileW, DeleteFileW, GetTempPathW, IsDebuggerPresent, VirtualProtect
```

```
DLL Name: ADVAPI32.dll
```

```
OpenProcessToken, GetTokenInformation, ConvertSidToStringSidW
```

```
Assessment: imports are consistent with a PyInstaller/Python runtime; these APIs alone do not prove malware.
```

6. String and PyInstaller Analysis

The file contains PyInstaller markers, python314.dll, PYZ.pyz, _MEIPASS, and application modules app, models and pdf_generator. Extracted app strings expose /api/login, /api/register, /api/bills, billing.db, a hard-coded secret and admin/admin123.

05 – PyInstaller archive evidence

```
$ strings -a BillingApp.exe | grep -E "PYINSTALLER|_MEIPASS|PYZ.pyz|python314.dll"
PYINSTALLER_SUPPRESS_SPLASH_SCREEN
_PYI_APPLICATION_HOME_DIR
_MEIPASS
PYZ archive entry not found in the TOC!
PYZ.pyz
python314.dll
```

```
$ extracted archive TOC (selected)
app
models
pdf_generator
base_library.zip
static_dist\assets\index-*.js
static_dist\index.html
```

04 – Extracted application strings

```
$ strings -a app_decompressed.bin | grep -E "api/|admin|billing|sqlite|127.0.0.1|SECRET_KEY"
/api/login      POST
/api/register   POST
/api/logout     POST
/api/me         GET
/api/bills      ...
/api/bills/<int:bill_id>/download
```

```
billing.db
billing-system-secret-key-2026
admin
admin123
```

```
Starting Flask server at http://127.0.0.1:
http://localhost:5173
http://localhost:5000
```

7. Network Indicator Review

No external URL/IP indicator was observed in the extracted application strings. Only localhost/127.0.0.1 application URLs were identified. This does not rule out runtime network activity.

```
06 - External network indicator check

$ grep -Eai "https?://[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" strings_ascii.txt

No external URL/IP indicators were found in the extracted application strings.

Observed URLs:
http://127.0.0.1:<PORT>
http://localhost:5173
http://localhost:5000

Interpretation: no hard-coded external C2 endpoint was observed statically.
Limitation: absence of a string does not prove absence of runtime network activity.
```

8. Key Findings

Finding	Evidence	Significance
PyInstaller package	PYZ.pyz, _MEIPASS, python314.dll	Packaging mechanism, not malicious by itself
Default credential	admin / admin123	High-risk deployment weakness
Hard-coded secret	billing-system-secret-key-2026	Replace before production
Local SQLite	billing.db	Potential sensitive data store
External C2	None observed statically	Dynamic analysis still required
Persistence	None observed statically	Dynamic analysis still required

9. Controlled Dynamic Analysis Procedure

In the isolated Windows VM: baseline process/network/filesystem state; start Wireshark; configure Procmon filters for BillingApp.exe; execute once; record child processes, file writes, registry changes, ports, DNS and outbound connections; export evidence; revert the snapshot. Linux-side commands include `sudo tcpdump -i eth0 -nn -w billingapp.pcap`, `sudo ss -ltnp` and `sudo lsof -i -n -P`.

10. Countermeasures

Remove hard-coded secrets and default credentials; use a secrets manager; enforce authorization on invoice/PDF endpoints; protect SQLite/PDF files with ACLs; restrict listening interfaces and firewall ports; patch dependencies; sign production builds; run under least privilege; enable EDR and centralized logging; isolate analysis samples from corporate networks.

11. Final Assessment

The static evidence is more consistent with a packaged billing/web application than conventional malware. Concrete application-security weaknesses exist and should be remediated. No external C2 or persistence was established statically. A final malware verdict requires controlled dynamic analysis.